



IILM
UNIVERSITY

PYTHON INTERNSHIP PROJECT REPORT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology (B.Tech.)

Submitted By

Ashwina Pandey

Roll No.: 25SCS1003004835

Branch: B.Tech. CSE

Section: 2CSE4

IILM University

Department of Computer Science and Engineering

Academic Year: 2025–2029

DECLARATION

I hereby declare that the project report titled “Python Internship Project” is a record of the work carried out by me as part of my internship and academic project work.

The project was developed to gain practical experience in Python programming, file handling, automation, logging, web development, version control, and cloud deployment.

I further declare that the information presented in this report is accurate to the best of my knowledge and represents the work completed during the internship.

Name: Ashwina Pandey

Roll No.: 25SCS1003004835

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **IILM University** and the Department of Computer Science and Engineering for providing me with the opportunity to undertake this Python internship project.

This internship provided me with an opportunity to apply programming concepts in practical situations and improve my understanding of software development.

Through this project, I gained practical knowledge of Python programming, virtual environments, file handling, automation, HTTP requests, logging, Flask web development, Git, GitHub, and cloud deployment.

I am thankful for the resources and learning opportunities that helped me complete this project successfully.

ABSTRACT

This project was developed as part of a Python internship to gain practical experience in Python programming and software development.

The project consists of four major deliverables: **Python Sandbox Setup, Directory Cleanup, Image Downloader, and Cron Scheduler & Logger.**

The Python Sandbox was created to establish and verify an isolated Python development environment. The Directory Cleanup utility was developed to automatically organize files into folders based on their file extensions. The Image Downloader reads image URLs from a text file and downloads the corresponding images using Python HTTP requests. The Cron Scheduler & Logger simulates scheduled task executions and records execution details, timestamps, and execution times in a log file.

A Flask-based web application was also developed to present the completed internship deliverables through a simple and organized interface. The project was maintained using Git and GitHub for version control and was deployed as a live web application using Render.

The project provided practical exposure to Python programming, automation, file management, HTTP requests, logging, web development, version control, debugging, testing, and cloud deployment.

TABLE OF CONTENTS

1. INTRODUCTION
2. OBJECTIVES
3. TECHNOLOGIES USED
4. PROJECT OVERVIEW
5. TASK 1 — PYTHON SANDBOX SETUP
6. TASK 2 — DIRECTORY CLEANUP
7. TASK 3 — IMAGE DOWNLOADER
8. TASK 4 — CRON SCHEDULER & LOGGER
9. WEB APPLICATION
10. GITHUB VERSION CONTROL
11. LIVE DEPLOYMENT
12. PROJECT STRUCTURE
13. SKILLS AND CONCEPTS LEARNED
14. TESTING AND RESULTS
15. CONCLUSION
16. FUTURE SCOPE
17. PROJECT LINKS

1. INTRODUCTION

Python is a high-level, interpreted programming language known for its simple syntax, readability, and wide range of applications. It is widely used in software development, automation, web development, data processing, scripting, and many other areas of computer science.

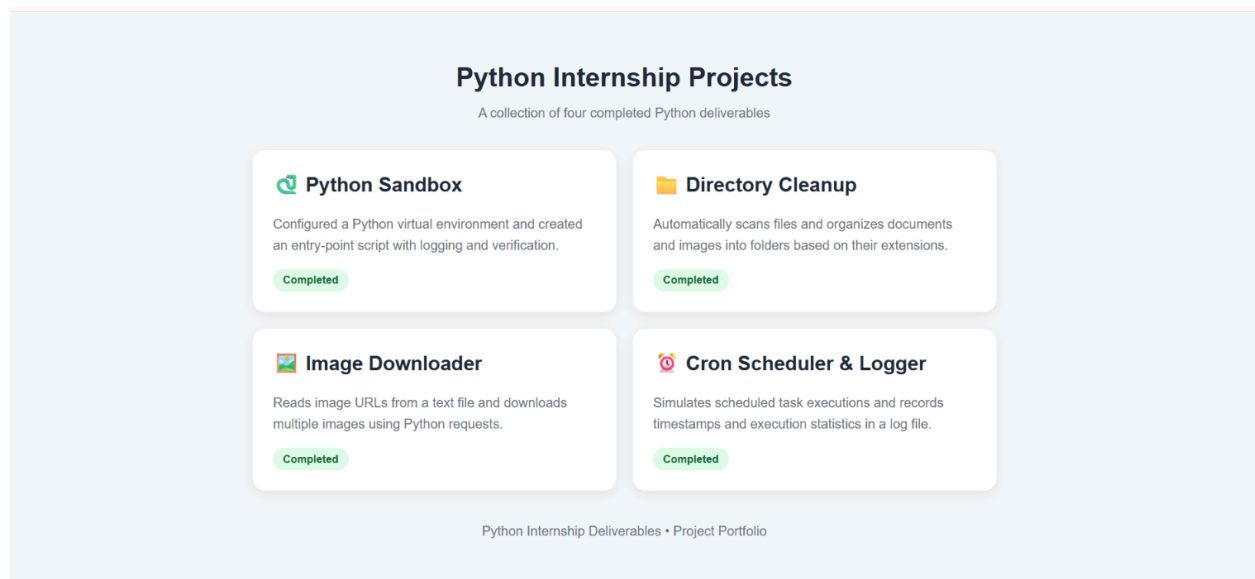
The purpose of this internship project was to gain practical experience by implementing different Python-based tasks and understanding the complete development workflow.

The project contains four major Python deliverables:

- Python Sandbox Setup
- Directory Cleanup
- Image Downloader
- Cron Scheduler & Logger

In addition to these utilities, a Flask web application was created to present the completed deliverables through a web interface.

The project was developed and tested using Visual Studio Code. Git and GitHub were used for version control, while Render was used to deploy the web application.



S

2. OBJECTIVES

The main objectives of the project were:

- To understand Python development environments.
- To create and use a Python virtual environment.
- To develop automated file-management functionality.
- To understand file and directory operations in Python.
- To work with HTTP requests and image downloading.
- To implement scheduling concepts.
- To maintain execution logs using Python's logging module.
- To develop a basic web application using Flask.
- To use Git and GitHub for source-code management.
- To deploy a Python web application online.
- To understand software testing and debugging.

- To gain practical experience in developing and documenting software projects.

3. TECHNOLOGIES USED

Technology	Purpose
Python	Main programming language
Flask	Web application development
Requests	Sending HTTP requests and downloading images
OS	File and directory operations
Shutil	Moving and organizing files
Logging	Recording execution information
Time	Measuring execution time
Git	Version control

GitHub Source-code hosting

Gunicorn Production WSGI server

Render Cloud deployment

Visual Studio Code Development environment

4. PROJECT OVERVIEW

The project consists of four main Python utilities and a Flask web interface.

4.1 Python Sandbox

The Python Sandbox establishes an isolated development environment and verifies that the Python environment is functioning correctly.

4.2 Directory Cleanup

The Directory Cleanup utility scans files and organizes them into appropriate folders according to their extensions.

4.3 Image Downloader

The Image Downloader reads image URLs from a text file and downloads the images using Python requests.

4.4 Cron Scheduler & Logger

The Cron Scheduler & Logger simulates scheduled task executions and records timestamps, execution status, and execution time in a log file.

4.5 Flask Web Application

A Flask web application was created to present all four completed deliverables in a simple project portfolio interface.

5. TASK 1 — PYTHON SANDBOX SETUP

Objective

The objective of this task was to create a Python development environment using a virtual environment and verify that Python was configured correctly.

Implementation

A virtual environment named `venv` was created using Python's built-in `venv` module.

An entry-point file named `main.py` was created to test the environment and demonstrate logging.

The project also contains a `requirements.txt` file to maintain the required Python dependencies.

Technologies Used

- Python
- `venv`
- pip
- Logging
- `requirements.txt`

Result

The virtual environment was successfully created and activated. The Python program executed successfully and displayed the required debug and verification messages.

```
main.py
1  import logging
2
3  logging.basicConfig(level=logging.DEBUG)
4
5  logging.debug("Python sandbox is running!")
6  logging.info("Python environment verified successfully.")
```

```
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python main.py
DEBUG:root:Python sandbox is running!
Python environment verified successfully.
```

6. TASK 2 — DIRECTORY CLEANUP LOGIC

Objective

The objective of this task was to create an automated file-organizing utility that sorts files into appropriate folders according to their file extensions.

Implementation

The `file_organizer.py` program scans the contents of a directory and identifies files based on their extensions.

The `os` module is used for directory and file operations, while the `shutil` module is used to move files.

Files such as `.txt` and `.pdf` are placed in the **Documents** folder, while `.jpg`, `.jpeg`, and `.png` files are placed in the **Images** folder.

Technologies Used

- Python
- OS module
- Shutil module
- Loops

- Conditional statements
- File extensions

Result

The program successfully detected different file types and automatically moved them into their respective folders.

```

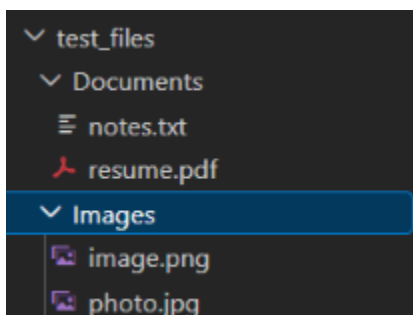
file_organizer.py > ...
1  import os
2  import shutil
3
4  # Folder containing the files we want to organize
5  source_folder = "test_files"
6
7  # File extension mappings
8  file_types = {
9      "Documents": [".txt", ".pdf", ".doc", ".docx"],
10     "Images": [".jpg", ".jpeg", ".png", ".gif"]
11 }
12
13 # Create the source folder if it doesn't exist
14 if not os.path.exists(source_folder):
15     os.makedirs(source_folder)
16     print("Created test_files folder.")
17     print("Put some .txt, .pdf, .jpg, or .png files inside it and run the program again.")
18 else:
19     # Scan all items in the folder
20     for filename in os.listdir(source_folder):
21
22         file_path = os.path.join(source_folder, filename)
23
24         # Skip folders
25         if not os.path.isfile(file_path):
26             continue
27
28         # Get the file extension
29         extension = os.path.splitext(filename)[1].lower()
30
31         # Find the correct category
32         target_folder = None

```

```

34         for folder, extensions in file_types.items():
35             if extension in extensions:
36                 target_folder = folder
37                 break
38
39         # Move the file if its type is recognized
40         if target_folder:
41             destination_folder = os.path.join(source_folder, target_folder)
42
43             # Create destination folder
44             if not os.path.exists(destination_folder):
45                 os.makedirs(destination_folder)
46
47             # Move the file
48             shutil.move(
49                 file_path,
50                 os.path.join(destination_folder, filename)
51             )
52
53             print(f"Moved {filename} → {target_folder}")
54         else:
55             print(f"Skipped {filename} (unknown file type)")
56
57     print("File organization completed!")

```



7. TASK 3 — IMAGE DOWNLOADER UTILITY

Objective

The objective of this task was to develop a Python utility that reads image URLs from a text file and downloads the corresponding images.

Implementation

Image URLs were stored in `image_urls.txt`.

The `image_downloader.py` program reads the URLs and uses the `requests` library to send HTTP GET requests.

The received image data is written to local files using binary mode.

The program also includes exception handling to manage network and file-related errors.

Multiple image URLs can be processed during a single execution.

Technologies Used

- Python
- Requests
- HTTP GET requests
- File handling
- Binary data
- Exception handling

Result

The program successfully downloaded multiple images and stored them inside the `downloaded_images` folder.

```
image_urls.txt
1 https://httpbin.org/image/jpeg
2 https://httpbin.org/image/png
```

```

image_downloader.py > ...
1  import os
2  import requests
3  from concurrent.futures import ThreadPoolExecutor
4
5  URL_FILE = "image_urls.txt"
6  OUTPUT_FOLDER = "downloaded_images"
7
8
9  def download_image(url):
10     try:
11         response = requests.get(url, timeout=10)
12         response.raise_for_status()
13
14         filename = url.split("/")[-1].split("?")[0]
15
16         if not filename or "." not in filename:
17             filename = f"image_{abs(hash(url))}.jpg"
18
19         file_path = os.path.join(OUTPUT_FOLDER, filename)
20
21         with open(file_path, "wb") as image_file:
22             image_file.write(response.content)
23
24         print(f"Downloaded: {filename}")
25
26     except requests.exceptions.RequestException as error:
27         print(f"Network error for {url}: {error}")
28
29     except OSError as error:
30         print(f"File error for {url}: {error}")
31
32     except Exception as error:
33         print(f"Unexpected error for {url}: {error}")
34

```

```

35
36 def main():
37     os.makedirs(OUTPUT_FOLDER, exist_ok=True)
38
39     try:
40         with open(URL_FILE, "r") as file:
41             urls = [line.strip() for line in file if line.strip()]
42     except FileNotFoundError:
43         print(f"Could not find {URL_FILE}")
44         return
45
46     print(f"Found {len(urls)} image URLs.")
47
48     with ThreadPoolExecutor(max_workers=3) as executor:
49         executor.map(download_image, urls)
50
51     print("Image downloading completed!")
52
53
54 if __name__ == "__main__":
55     main()

```

```

(venv) aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python image_downloader.py
Found 3 image URLs.
Network error for https://picsum.photos/300/200: 503 Server Error: Service Unavailable for url: https://picsum.photos/300/200
Network error for https://picsum.photos/500/400: HTTPConnectionPool(host='picsum.photos', port=443): Read timed out. (read timeout=10)
Network error for https://picsum.photos/400/300: HTTPConnectionPool(host='picsum.photos', port=443): Read timed out. (read timeout=10)
Image downloading completed!
(venv) aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python image_downloader.py
Found 2 image URLs.
Downloaded: image_8361623383926274516.jpg
Downloaded: image_4139798086594451692.jpg
Image downloading completed!

```

8. TASK 4 — CRON SCHEDULER & LOGGER

Objective

The objective of this task was to simulate scheduled task execution and maintain a persistent log of the executions.

Implementation

The `scheduler_logger.py` program simulates multiple scheduled task runs.

Python's `logging` module is used to record events, timestamps, task status, and execution time.

The information is stored in the `scheduler.log` file, creating a persistent audit trail of the executions.

Technologies Used

- Python
- Logging module
- Time module
- Date and time
- File logging
- Scheduling concepts

Result

The program successfully executed multiple simulated runs and recorded the execution details in `scheduler.log`.

```

scheduler_logger.py > ...
1 import logging
2 import time
3 from datetime import datetime
4
5
6 # Configure logging
7 logging.basicConfig(
8     filename="scheduler.log",
9     level=logging.INFO,
10    format="%(asctime)s - %(levelname)s - %(message)s"
11 )
12
13
14 def scheduled_task():
15     """Simulate an automated task."""
16     start_time = time.time()
17
18     logging.info("Scheduled task started.")
19
20     # Simulated task work
21     time.sleep(1)
22
23     end_time = time.time()
24     execution_time = end_time - start_time
25
26     logging.info(
27         f"Scheduled task completed successfully. "
28         f"Execution time: {execution_time:.2f} seconds"
29     )
30
31     print(
32         f"Task executed at {datetime.now().strftime('%Y-%m-%d %H:%M:%S')} "
33         f"in {execution_time:.2f} seconds."
34     )

```

```

37 def main():
38     print("Cron Scheduler & Logger started.")
39
40     # Run the scheduled task three times
41     for run_number in range(1, 4):
42         logging.info(f"Run #{run_number} initiated.")
43
44         scheduled_task()
45
46         logging.info(f"Run #{run_number} finished.")
47
48         # Short delay to simulate scheduling
49         if run_number < 3:
50             time.sleep(2)
51
52     logging.info("All scheduled runs completed.")
53     print("All scheduled runs completed.")
54
55
56 if __name__ == "__main__":
57     main()

```

```

scheduler.log
1 2026-08-31 13:00:24,800 - INFO - Run #1 initiated.
2 2026-08-31 13:00:24,801 - INFO - Scheduled task started.
3 2026-08-31 13:00:25,802 - INFO - Scheduled task completed successfully. Execution time: 1.00 seconds
4 2026-08-31 13:00:25,802 - INFO - Run #1 finished.
5 2026-08-31 13:00:28,397 - INFO - Run #2 initiated.
6 2026-08-31 13:00:28,398 - INFO - Scheduled task started.
7 2026-08-31 13:00:29,400 - INFO - Scheduled task completed successfully. Execution time: 1.00 seconds
8 2026-08-31 13:00:29,402 - INFO - Run #2 finished.
9 2026-08-31 13:00:31,404 - INFO - Run #3 initiated.
10 2026-08-31 13:00:31,405 - INFO - Scheduled task started.
11 2026-08-31 13:00:32,408 - INFO - Scheduled task completed successfully. Execution time: 1.00 seconds
12 2026-08-31 13:00:32,409 - INFO - Run #3 finished.
13 2026-08-31 13:00:32,410 - INFO - All scheduled runs completed.

```

```

(venv) aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python scheduler_logger.py
Cron Scheduler & Logger started.
Task executed at 2026-08-31 13:00:25 in 1.00 seconds.
Task executed at 2026-08-31 13:00:29 in 1.00 seconds.
Task executed at 2026-08-31 13:00:32 in 1.00 seconds.
All scheduled runs completed.

```

9. WEB APPLICATION

Objective

A simple web application was created to present the completed internship deliverables through a user-friendly interface.

Implementation

The web application was developed using **Flask**.

The application contains a homepage displaying the four completed Python tasks along with their completion status.

The application was tested locally before being deployed online.

Technologies Used

- Python
- Flask
- HTML
- CSS
- Unicorn

```
app.py > -
1  from flask import Flask
2
3  app = Flask(__name__)
4
5
6  @app.route("/")
7  def home():
8      return """
9      <!DOCTYPE html>
10     <html>
11     <head>
12         <title>Python Internship Projects</title>
13         <style>
14             body {
15                 font-family: Arial, sans-serif;
16                 background: #f4f7fb;
17                 margin: 0;
18                 padding: 40px;
19                 color: #1f2937;
20             }
21
22             .container {
23                 max-width: 900px;
24                 margin: auto;
25             }
26
27             h1 {
28                 text-align: center;
29                 margin-bottom: 10px;
30             }
31
32             .subtitle {
33                 text-align: center;
34                 color: #6b7280;
35                 margin-bottom: 35px;
36             }
37         </style>
38     </head>
39     <body>
40         <div class="container">
41             <h1>Python Internship Projects</h1>
42             <div class="subtitle">
43                 <p>A simple web application was created to present the completed internship deliverables through a user-friendly interface.</p>
44             </div>
45         </div>
46     </body>
47     </html>
48     """
```

```

38     .grid {
39         display: grid;
40         grid-template-columns: repeat(2, 1fr);
41         gap: 20px;
42     }
43
44     .card {
45         background: white;
46         padding: 25px;
47         border-radius: 15px;
48         box-shadow: 0 4px 15px rgba(0,0,0,0.08);
49     }
50
51     .card h2 {
52         margin-top: 0;
53     }
54
55     .card p {
56         color: #6b7280;
57         line-height: 1.6;
58     }
59
60     .status {
61         display: inline-block;
62         background: #dcfce7;
63         color: #166534;
64         padding: 6px 12px;
65         border-radius: 20px;
66         font-size: 13px;
67         font-weight: bold;
68     }
69

```

```

70     footer {
71         text-align: center;
72         margin-top: 40px;
73         color: #6b7280;
74     }
75
76     @media (max-width: 700px) {
77         .grid {
78             grid-template-columns: 1fr;
79         }
80     }
81 </style>
82 </head>
83
84 <body>
85     <div class="container">
86
87         <h1>Python Internship Projects</h1>
88
89         <p class="subtitle">
90             A collection of four completed Python deliverables
91         </p>
92
93         <div class="grid">
94
95             <div class="card">
96                 <h2>🐍 Python Sandbox</h2>
97                 <p>
98                     Configured a Python virtual environment and created
99                     an entry-point script with logging and verification.
100                 </p>
101                 <span class="status">Completed</span>
102             </div>

```

```

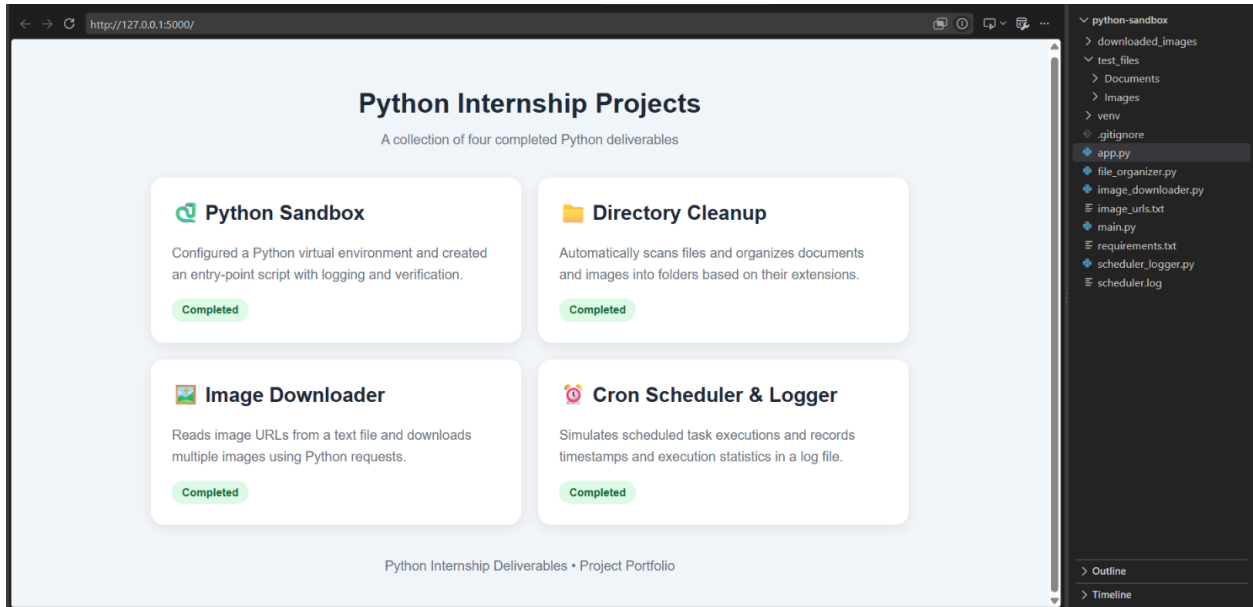
104             <div class="card">
105                 <h2>📁 Directory Cleanup</h2>
106                 <p>
107                     Automatically scans files and organizes documents
108                     and images into folders based on their extensions.
109                 </p>
110                 <span class="status">Completed</span>
111             </div>
112
113             <div class="card">
114                 <h2>📄 Image Downloader</h2>
115                 <p>
116                     Reads image URLs from a text file and downloads
117                     multiple images using Python requests.
118                 </p>
119                 <span class="status">Completed</span>
120             </div>
121
122             <div class="card">
123                 <h2>🕒 Cron Scheduler & Logger</h2>
124                 <p>
125                     Simulates scheduled task executions and records
126                     timestamps and execution statistics in a log file.
127                 </p>
128                 <span class="status">Completed</span>
129             </div>
130
131         </div>
132
133         <footer>
134             Python Internship Deliverables • Project Portfolio
135         </footer>

```

```

137     </div>
138 </body>
139 </html>
140 """
141
142
143 if __name__ == "__main__":
144     app.run(host="0.0.0.0", port=5000)

```



10. GITHUB VERSION CONTROL

The completed project was uploaded to a public GitHub repository.

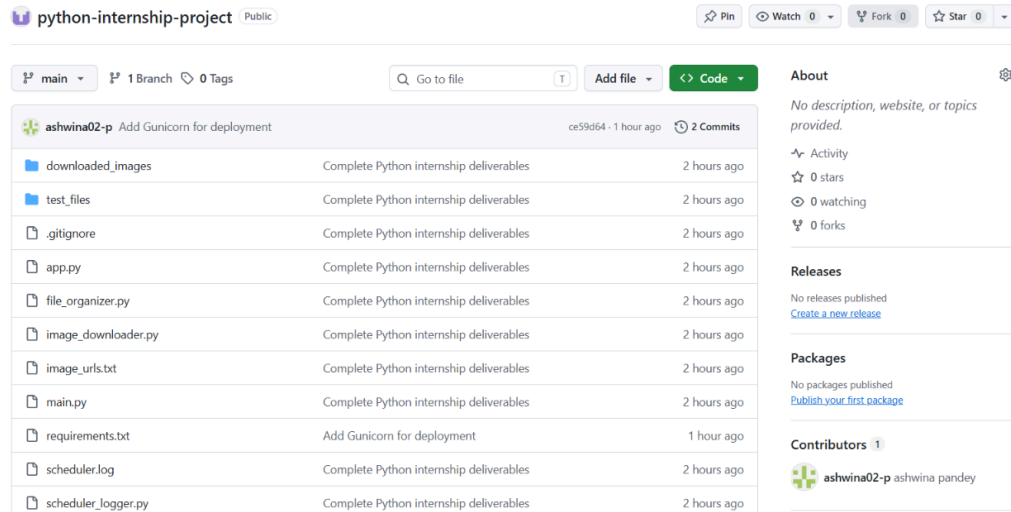
Git was used to initialize the local repository, track project files, create commits, and push the project to GitHub.

A `.gitignore` file was also added to prevent unnecessary files such as the Python virtual environment from being uploaded.

Repository

GitHub Repository:

<https://github.com/ashwina0202-ai/python-internship-project>



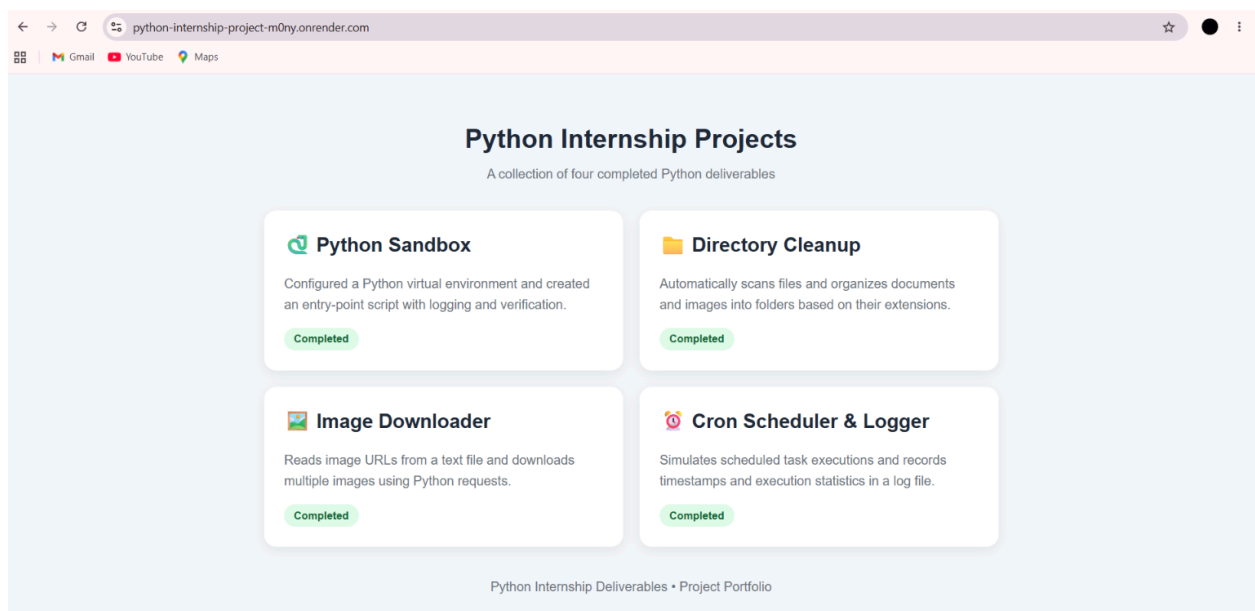
11. LIVE DEPLOYMENT

The Flask web application was deployed using Render.

The application was configured with the required Python dependencies and Gunicorn as the production server.

Live Application

<https://python-internship-project-m0ny.onrender.com>



12. PROJECT STRUCTURE

The main project files are organized as follows:

```
python-internship-project/
|
|— app.py
|— main.py
|— file_organizer.py
|— image_downloader.py
|— image_urls.txt
|— scheduler_logger.py
|— scheduler.log
|— requirements.txt
|— .gitignore
|
|— downloaded_images/
|— test_files/
```

The `venv` directory was used locally for the Python virtual environment and was excluded from GitHub using `.gitignore`.

13. SKILLS AND CONCEPTS LEARNED

During the completion of these tasks, I gained practical experience with:

- Python programming
- Virtual environments
- Package management
- File and directory handling
- File extensions
- Automation
- HTTP requests
- Binary file handling
- Exception handling
- Logging
- Scheduling concepts

- Flask
 - Git
 - GitHub
 - Cloud deployment
 - Unicorn
-

14. TESTING AND RESULTS

Each task was tested locally before being considered complete.

The directory cleanup utility was tested using sample files and successfully organized them into appropriate folders.

The image downloader successfully retrieved images from the provided URLs.

The scheduler successfully executed multiple runs and generated timestamped log entries.

The Flask application was successfully tested locally and subsequently deployed online.

15. CONCLUSION

The Python Internship Project provided valuable practical experience in Python programming and software development.

All four major deliverables were successfully implemented and tested. The Python Sandbox demonstrated environment setup and logging, while the Directory Cleanup utility demonstrated automated file organization.

The Image Downloader provided practical experience with HTTP requests and file downloading. The Cron Scheduler & Logger demonstrated task execution, timestamps, execution-time measurement, and persistent logging.

A Flask-based web application was also developed to present the completed deliverables. Git and GitHub were used for version control, and Render was used for live deployment.

Overall, the project helped strengthen practical skills in Python programming, automation, file handling, web development, debugging, testing, version control, and cloud deployment.

16. FUTURE SCOPE

The project can be further enhanced with additional features and improvements.

Possible future enhancements include:

- Adding a database to store task execution history.
- Adding user authentication to the web application.
- Providing interactive controls for running individual utilities.
- Adding real-time scheduler monitoring.
- Adding email or notification alerts.
- Supporting additional file formats in the Directory Cleanup utility.
- Adding improved image validation and download controls.
- Adding automated unit tests.
- Improving the web interface with additional interactive features.
- Adding a dashboard for displaying task execution statistics.
- Adding more advanced error reporting.
- Providing downloadable reports of completed tasks.

These improvements could make the application more interactive, scalable, and suitable for real-world use.

17. PROJECT LINKS

GitHub Repository

<https://github.com/ashwina0202-ai/python-internship-project>

Live Application

<https://python-internship-project-m0ny.onrender.com>

INTERNSHIP CERTIFICATE

